



Complete React Basics Topics Guide for Beginners ⚡

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) 📁 | [X: @_19_neeraj](#) 🐦 | [GitHub: Neeraj05042001](#) 🧑 |

Welcome to your comprehensive guide to mastering React fundamentals! This guide covers everything you need to know to start Learning & building React applications.



Table of Contents

1. [Quick React Setup](#)
2. [JSX \(JavaScript XML\)](#)
3. [Vite Dev Server](#)
4. [Components](#)
5. [Styling](#)
6. [Props](#)
7. [Lists & Keys](#)
8. [Event Handling](#)
9. [Hooks](#)
10. [State & useState Hook](#)
11. [Conditional Rendering](#)
12. [Component Composition](#)
13. [Controlled Components](#)
14. [Using Forms](#)
15. [Basic Validation](#)
16. [Form Submission](#)
17. [Form UI State](#)
18. [Prop Drilling & Lifting Up State](#)



Quick React Setup

What is React?

React is a JavaScript library for building user interfaces. It lets you create reusable components that manage their own state.

Setting Up with Vite

Vite is the modern, lightning-fast build tool for React projects.

```
# Create a new React project
npm create vite@latest my-react-app -- --template react

# Navigate to project folder
cd my-react-app

# Install dependencies
npm install

# Start development server
npm run dev
```

Project Structure

```
my-react-app/
├── node_modules/
├── public/
├── src/
│   ├── App.jsx
│   ├── App.css
│   └── main.jsx
├── index.html
├── package.json
└── vite.config.js
```

Key Files:

- `main.jsx` - Entry point of your React app
- `App.jsx` - Root component
- `index.html` - HTML template
- `vite.config.js` - Vite configuration

JSX (JavaScript XML)

What is JSX?

JSX is a syntax extension that lets you write HTML-like code in JavaScript. It makes React components more readable and expressive.

🌟 JSX Rules

1. Return a Single Root Element

```
// ❌ Wrong - Multiple root elements
function App() {
  return (
    <h1>Hello</h1>
    <p>World</p>
  );
}
```

```
// ✅ Correct - Wrapped in a parent element
function App() {
  return (
    <div>
      <h1>Hello</h1>
      <p>World</p>
    </div>
  );
}
```

```
// ✅ Also Correct - Using Fragment
function App() {
  return (
    <>
      <h1>Hello</h1>
      <p>World</p>
    </>
  );
}
```

2. Close All Tags

```
// ❌ Wrong

<input type="text">
```

```
// ✅ Correct

<input type="text" />
```

3. Use camelCase for Attributes

```
// ❌ Wrong
<div class="container" onclick="handleClick()">

// ✅ Correct
<div className="container" onClick={handleClick}>
```

4. Embed JavaScript with Curly Braces

```
function Greeting() {
  const name = "John";
  const age = 25;

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>You are {age} years old</p>
      <p>Next year you'll be {age + 1}</p>
    </div>
  );
}
```

🎨 JSX vs HTML Differences

HTML	JSX
class	className
for	htmlFor
onclick	onClick
onchange	onChange
tabindex	tabIndex

⚡ Vite Dev Server

What is Vite?

Vite is a modern build tool that provides:

- ⚡ Lightning-fast hot module replacement (HMR)
- 🚀 Instant server start
- 📦 Optimized builds

-  Easy configuration

Common Commands

```
# Start development server  
npm run dev
```

```
# Build for production  
npm run build
```

```
# Preview production build  
npm run preview
```

Hot Module Replacement (HMR)

When you save a file, Vite instantly updates your browser without refreshing the page. Your app state is preserved!

Components

What are Components?

Components are reusable, independent pieces of UI. Think of them as custom HTML elements.

Types of Components

1. Function Components (Modern & Recommended)

```
function Welcome() {  
  return <h1>Hello, World!</h1>;  
}
```

2. Arrow Function Components

```
const Welcome = () => {  
  return <h1>Hello, World!</h1>;  
};
```

```
// Shorter syntax for simple returns  
const Welcome = () => <h1>Hello, World!</h1>;
```

Component Best Practices

✓ Component Naming

- Always use **PascalCase** for component names
- Names should be descriptive

// ✓ Good

```
function UserProfile() { }  
function ShoppingCart() { }
```

// ✗ Bad

```
function userprofile() { }  
function component1() { }
```

📁 File Organization

```
src/  
├── components/  
│   ├── Header.jsx  
│   ├── Footer.jsx  
│   ├── Button.jsx  
│   └── UserCard.jsx  
├── App.jsx  
└── main.jsx
```

🔗 Using Components

```
// Define components  
function Header() {  
  return <header><h1>My Website</h1></header>;  
}  
  
function Content() {  
  return <main><p>Welcome to my site!</p></main>;  
}  
  
function Footer() {  
  return <footer><p>&copy; 2025</p></footer>;  
}  
  
// Use components in App  
function App() {  
  return (  
    <div>  
      <Header />  
      <Content />  
      <Footer />  
    </div>  
  );  
}
```

```
);  
}
```

Styling

1. Inline Styles

```
function Button() {  
  const buttonStyle = {  
    backgroundColor: 'blue',  
    color: 'white',  
    padding: '10px 20px',  
    borderRadius: '5px',  
    border: 'none'  
  };  
  
  return <button style={buttonStyle}>Click Me</button>;  
}  
  
// Or directly  
function Button() {  
  return (  
    <button style={{  
      backgroundColor: 'blue',  
      color: 'white'  
    }}>  
      Click Me  
    </button>  
  );  
}
```

2. CSS Files

```
/* Button.css */  
.btn {  
  background-color: blue;  
  color: white;  
  padding: 10px 20px;  
  border-radius: 5px;  
  border: none;  
}  
  
.btn:hover {  
  background-color: darkblue;  
}
```

```
// Button.jsx
import './Button.css';

function Button() {
  return <button className="btn">Click Me</button>;
}
```

3. CSS Modules (Scoped Styles)

```
/* Button.module.css */
.btn {
  background-color: blue;
  color: white;
}
```

```
// Button.jsx
import styles from './Button.module.css';

function Button() {
  return <button className={styles.btn}>Click Me</button>;
}
```

4. Conditional Styling

```
function Button({ isPrimary }) {
  const buttonClass = isPrimary ? 'btn-primary' : 'btn-secondary';

  return <button className={buttonClass}>Click Me</button>;
}

// With multiple classes
function Card({ isActive, isLarge }) {
  const classes = `card ${isActive ? 'active' : ''} ${isLarge ? 'large' : ''}`;

  return <div className={classes}>Card Content</div>;
}
```

Props

What are Props?

Props (properties) are arguments passed to components. They let you customize components and pass data from parent to child.

Basic Props Usage

```
// Parent component
function App() {
  return (
    <div>
      <Welcome name="Alice" age={25} />
      <Welcome name="Bob" age={30} />
    </div>
  );
}

// Child component
function Welcome(props) {
  return (
    <div>
      <h1>Hello, {props.name}!</h1>
      <p>You are {props.age} years old</p>
    </div>
  );
}
```

Props Destructuring

```
// Instead of props.name, props.age
function Welcome({ name, age }) {
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>You are {age} years old</p>
    </div>
  );
}
```

Different Types of Props

```
function UserCard({
  name,           // String
  age,            // Number
  isOnline,       // Boolean
  hobbies,        // Array
  address,        // Object
  onClick        // Function
}) {
```

```

return (
  <div onClick={onClick}>
    <h2>{name}</h2>
    <p>Age: {age}</p>
    <p>Status: {isOnline ? '🟢 Online' : '🔴 Offline'}</p>
    <p>Hobbies: {hobbies.join(', ')}</p>
    <p>City: {address.city}</p>
  </div>
);
}

// Usage
<UserCard
  name="Alice"
  age={25}
  isOnline={true}
  hobbies={['Reading', 'Gaming']}
  address={{ city: 'New York', country: 'USA' }}
  onClick={() => console.log('Card clicked!')}
/>

```

★ Default Props

```

function Button({ text = 'Click Me', color = 'blue' }) {
  return (
    <button style={{ backgroundColor: color }}>
      {text}
    </button>
  );
}

// Usage
<Button /> // Uses defaults
<Button text="Submit" color="green" /> // Custom props

```

👶 Children Prop

```

function Card({ children }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}

// Usage
<Card>
  <h2>Title</h2>

```

```
<p>This is the content inside the card</p>
</Card>
```

🚫 Props are Read-Only!

```
// ❌ Wrong - Never modify props
function Welcome(props) {
  props.name = 'Something else'; // Don't do this!
  return <h1>Hello, {props.name}</h1>;
}

// ✅ Correct - Props are immutable
function Welcome({ name }) {
  return <h1>Hello, {name}</h1>;
}
```

Lists & Keys

Rendering Lists

```
function FruitList() {
  const fruits = ['Apple', 'Banana', 'Orange', 'Mango'];

  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Why Keys Matter

Keys help React identify which items have changed, been added, or removed. This improves performance.

```
// ❌ Bad - Using index as key (not recommended for dynamic lists)
{items.map((item, index) => (
  <div key={index}>{item}</div>
))}

// ✅ Good - Using unique ID
{items.map((item) => (
```

```
    <div key={item.id}>{item.name}</div>
  )})
}
```

Rendering Object Arrays

```
function UserList() {
  const users = [
    { id: 1, name: 'Alice', age: 25 },
    { id: 2, name: 'Bob', age: 30 },
    { id: 3, name: 'Charlie', age: 35 }
  ];

  return (
    <div>
      {users.map((user) => (
        <div key={user.id} className="user-card">
          <h3>{user.name}</h3>
          <p>Age: {user.age}</p>
        </div>
      ))}
    </div>
  );
}
```

Filtering Lists

```
function ProductList() {
  const products = [
    { id: 1, name: 'Laptop', price: 1000, inStock: true },
    { id: 2, name: 'Phone', price: 500, inStock: false },
    { id: 3, name: 'Tablet', price: 300, inStock: true }
  ];

  const availableProducts = products.filter(product => product.inStock);

  return (
    <div>
      <h2>Available Products</h2>
      {availableProducts.map((product) => (
        <div key={product.id}>
          <h3>{product.name}</h3>
          <p>${product.price}</p>
        </div>
      ))}
    </div>
  );
}
```



Event Handling

Basic Event Handling

```
function Button() {
  function handleClick() {
    alert('Button clicked!');
  }

  return <button onClick={handleClick}>Click Me</button>;
}

// Or with arrow function
function Button() {
  return (
    <button onClick={() => alert('Clicked!')}>
      Click Me
    </button>
  );
}
```



Common Events

```
function EventExamples() {
  return (
    <div>
      {/* Click Events */}
      <button onClick={() => console.log('Clicked!')}>Click</button>
      <button onDoubleClick={() => console.log('Double clicked!')}>Double Click</button>

      {/* Mouse Events */}
      <div
        onMouseEnter={() => console.log('Mouse entered')}
        onMouseLeave={() => console.log('Mouse left')}
      >
        Hover over me
      </div>

      {/* Input Events */}
      <input
        onChange={(e) => console.log(e.target.value)}
        placeholder="Type something"
      />

      {/* Form Events */}
      <form onSubmit={(e) => {
        e.preventDefault();
        console.log('Form submitted');
      }}>
```

```

        <button type="submit">Submit</button>
    </form>

    { /* Keyboard Events */ }
    <input
        onKeyDown={(e) => console.log('Key pressed:', e.key)}
        placeholder="Press any key"
    />
</div>
);
}

```

Event Object

```

function Input() {
    function handleChange(event) {
        console.log('Value:', event.target.value);
        console.log('Name:', event.target.name);
        console.log('Type:', event.type);
    }

    return (
        <input
            name="username"
            onChange={handleChange}
            placeholder="Enter username"
        />
    );
}

```

Passing Arguments to Event Handlers

```

function ButtonList() {
    function handleClick(id, name) {
        console.log(`Clicked button ${id}: ${name}`);
    }

    return (
        <div>
            <button onClick={() => handleClick(1, 'First')}>Button 1</button>
            <button onClick={() => handleClick(2, 'Second')}>Button 2</button>
            <button onClick={() => handleClick(3, 'Third')}>Button 3</button>
        </div>
    );
}

```

Preventing Default Behavior

```
function Form() {
  function handleSubmit(event) {
    event.preventDefault(); // Prevents page reload
    console.log('Form submitted without reload!');
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Hooks

What are Hooks?

Hooks are special functions that let you use React features in function components. They were introduced in React 16.8.

Rules of Hooks

1.  Only call hooks at the top level (not inside loops, conditions, or nested functions)
2.  Only call hooks in React function components or custom hooks
3.  Hook names must start with "use"

```
//  Wrong
function Component() {
  if (condition) {
    const [state, setState] = useState(0); // Don't do this!
  }
}
```

```
//  Correct
function Component() {
  const [state, setState] = useState(0);

  if (condition) {
    // Use state here
  }
}
```

Common Built-in Hooks

- `useState` - Manage component state
- `useEffect` - Perform side effects
- `useContext` - Access context values
- `useRef` - Reference DOM elements
- `useMemo` - Memoize expensive calculations
- `useCallback` - Memoize functions

State & `useState` Hook

What is State?

State is data that changes over time in your component. When state changes, React re-renders the component.

Basic `useState`

```
import { useState } from 'react';

function Counter() {
  // Declare state variable
  const [count, setCount] = useState(0);
  //      ^state ^updater ^initial value

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>
      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}
```

Multiple State Variables

```
function UserProfile() {
  const [name, setName] = useState('');
  const [age, setAge] = useState(0);
}
```



```

const [email, setEmail] = useState('');

return (
  <div>
    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
      placeholder="Name"
    />
    <input
      type="number"
      value={age}
      onChange={(e) => setAge(Number(e.target.value))}
      placeholder="Age"
    />
    <input
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      placeholder="Email"
    />

    <h3>Profile:</h3>
    <p>Name: {name}</p>
    <p>Age: {age}</p>
    <p>Email: {email}</p>
  </div>
);
}

```

State with Objects

```

function UserForm() {
  const [user, setUser] = useState({
    name: '',
    email: '',
    age: 0
  });

  function handleChange(e) {
    const { name, value } = e.target;

    setUser(prevUser => ({
      ...prevUser,      // Spread previous state
      [name]: value      // Update specific field
    }));
  }

  return (
    <div>
      <input

```

```

        name="name"
        value={user.name}
        onChange={handleChange}
      />
      <input
        name="email"
        value={user.email}
        onChange={handleChange}
      />
      <input
        name="age"
        type="number"
        value={user.age}
        onChange={handleChange}
      />
    </div>
  );
}

```

State with Arrays

```

function TodoList() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');

  function addTodo() {
    setTodos([...todos, input]);
    setInput('');
  }

  function removeTodo(index) {
    setTodos(todos.filter((_, i) => i !== index));
  }

  return (
    <div>
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
      />
      <button onClick={addTodo}>Add</button>

      <ul>
        {todos.map((todo, index) => (
          <li key={index}>
            {todo}
            <button onClick={() => removeTodo(index)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```
);  
}
```

⚡ Functional Updates

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  // ❌ May not work correctly with rapid updates  
  function incrementBad() {  
    setCount(count + 1);  
    setCount(count + 1);  
    setCount(count + 1);  
    // count only increases by 1!  
  }  
  
  // ✅ Works correctly with functional update  
  function incrementGood() {  
    setCount(prev => prev + 1);  
    setCount(prev => prev + 1);  
    setCount(prev => prev + 1);  
    // count increases by 3!  
  }  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={incrementGood}>+3</button>  
    </div>  
  );  
}
```

🔗 Conditional Rendering

Using if/else

```
function Greeting({ isLoggedIn }) {  
  if (isLoggedIn) {  
    return <h1>Welcome back!</h1>;  
  } else {  
    return <h1>Please sign in.</h1>;  
  }  
}
```

✨ Ternary Operator

```
function Greeting({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (
        <h1>Welcome back!</h1>
      ) : (
        <h1>Please sign in.</h1>
      )}
    </div>
  );
}
```

Logical && Operator

```
function Notification({ hasNewMessages, messageCount }) {
  return (
    <div>
      <h1>Dashboard</h1>
      {hasNewMessages && (
        <p>You have {messageCount} new messages!</p>
      )}
    </div>
  );
}
```

Multiple Conditions

```
function UserStatus({ status }) {
  return (
    <div>
      {status === 'online' && <span>🟢 Online</span>}
      {status === 'offline' && <span>🔴 Offline</span>}
      {status === 'away' && <span>🟡 Away</span>}
    </div>
  );
}
```

Conditional Styling

```
function Button({ isPrimary, isDisabled }) {
  return (
    <button
      className={`btn ${isPrimary ? 'btn-primary' : 'btn-secondary'}`}
      disabled={isDisabled}
      style={{
        opacity: isDisabled ? 0.5 : 1,
      }}
    >
```

```

        cursor: isDisabled ? 'not-allowed' : 'pointer'
      }}
    >
      Click Me
    </button>
  );
}

```

Component Composition

What is Component Composition?

Component composition is building complex UIs by combining simpler components together.

Container and Children Pattern

```

function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-content">
        {children}
      </div>
    </div>
  );
}

function App() {
  return (
    <div>
      <Card title="User Profile">
        <p>Name: John Doe</p>
        <p>Email: john@example.com</p>
      </Card>

      <Card title="Settings">
        <button>Edit Profile</button>
        <button>Change Password</button>
      </Card>
    </div>
  );
}

```

Layout Components

```

function Layout({ children }) {
  return (
    <div className="layout">
      <Header />
      <main>{children}</main>
      <Footer />
    </div>
  );
}

function App() {
  return (
    <Layout>
      <h1>Welcome to My App</h1>
      <p>This is the main content</p>
    </Layout>
  );
}

```

Composition vs Props

```

// ❌ Too many props (prop drilling)
function Dialog({ showCloseButton, closeButtonText, closeButtonColor, title, content }) {
  return (
    <div>
      <h2>{title}</h2>
      <p>{content}</p>
      {showCloseButton && (
        <button style={{ color: closeButtonColor }}>
          {closeButtonText}
        </button>
      )}
    </div>
  );
}

// ✅ Better with composition
function Dialog({ children, title }) {
  return (
    <div>
      <h2>{title}</h2>
      {children}
    </div>
  );
}

// Usage
<Dialog title="Welcome">
  <p>This is the content</p>

```

```
<button className="close-btn">Close</button>
</Dialog>
```



Controlled Components

What are Controlled Components?

Controlled components are form elements whose values are controlled by React state.



Controlled Input

```
function ControlledInput() {
  const [value, setValue] = useState('');

  return (
    <div>
      <input
        type="text"
        value={value}
        onChange={(e) => setValue(e.target.value)}
        placeholder="Type something..."
      />
      <p>You typed: {value}</p>
    </div>
  );
}
```



Multiple Controlled Inputs

```
function Form() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: ''
  });

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  }

  return (
    <form>
```

```

    <input
      name="username"
      value={formData.username}
      onChange={handleChange}
      placeholder="Username"
    />
    <input
      name="email"
      type="email"
      value={formData.email}
      onChange={handleChange}
      placeholder="Email"
    />
    <input
      name="password"
      type="password"
      value={formData.password}
      onChange={handleChange}
      placeholder="Password"
    />
  </form>
);
}

```

✓ Controlled Checkbox

```

function CheckboxExample() {
  const [isChecked, setIsChecked] = useState(false);

  return (
    <label>
      <input
        type="checkbox"
        checked={isChecked}
        onChange={(e) => setIsChecked(e.target.checked)}
      />
      Accept Terms and Conditions
    </label>
  );
}

```

📱 Controlled Radio Buttons

```

function RadioExample() {
  const [selectedOption, setSelectedOption] = useState('');

  return (
    <div>
      <label>

```



```

    <input
      type="radio"
      value="option1"
      checked={selectedOption === 'option1'}
      onChange={(e) => setSelectedOption(e.target.value)}
    />
    Option 1
  </label>

  <label>
    <input
      type="radio"
      value="option2"
      checked={selectedOption === 'option2'}
      onChange={(e) => setSelectedOption(e.target.value)}
    />
    Option 2
  </label>

  <p>Selected: {selectedOption}</p>
</div>
);
}

```

Controlled Select

```

function SelectExample() {
  const [country, setCountry] = useState('');

  return (
    <select
      value={country}
      onChange={(e) => setCountry(e.target.value)}
    >
      <option value="">Select Country</option>
      <option value="usa">USA</option>
      <option value="uk">UK</option>
      <option value="canada">Canada</option>
    </select>
  );
}

```

Using Forms

Complete Form Example

```

import { useState } from 'react';

function RegistrationForm() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: '',
    confirmPassword: '',
    agreeToTerms: false
  });

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log('Form submitted:', formData);
  }

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label>Username:</label>
        <input
          name="username"
          value={formData.username}
          onChange={handleChange}
          required
        />
      </div>

      <div>
        <label>Email:</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
          required
        />
      </div>

      <div>
        <label>Password:</label>
        <input
          type="password"
          name="password"

```

```

        value={formData.password}
        onChange={handleChange}
        required
      />
    </div>

    <div>
      <label>Confirm Password:</label>
      <input
        type="password"
        name="confirmPassword"
        value={formData.confirmPassword}
        onChange={handleChange}
        required
      />
    </div>

    <div>
      <label>
        <input
          type="checkbox"
          name="agreeToTerms"
          checked={formData.agreeToTerms}
          onChange={handleChange}
          required
        />
        I agree to the terms and conditions
      </label>
    </div>

    <button type="submit">Register</button>
  </form>
);
}

```

Form with Multiple Input Types

```

function SurveyForm() {
  const [survey, setSurvey] = useState({
    name: '',
    age: '',
    gender: '',
    interests: [],
    feedback: '',
    rating: '5'
  });

  function handleCheckboxChange(e) {
    const { value, checked } = e.target;
    setSurvey(prev => ({
      ...prev,

```

```

      interests: checked
      ? [...prev.interests, value]
      : prev.interests.filter(item => item !== value)
    }));
  }
}

```

```

function handleChange(e) {
  const { name, value } = e.target;
  setSurvey(prev => ({
    ...prev,
    [name]: value
  }));
}

```

```

return (
  <form>
    {/* Text Input */}
    <input
      name="name"
      value={survey.name}
      onChange={handleChange}
      placeholder="Your Name"
    />

    {/* Number Input */}
    <input
      type="number"
      name="age"
      value={survey.age}
      onChange={handleChange}
      placeholder="Your Age"
    />

    {/* Radio Buttons */}
    <div>
      <label>
        <input
          type="radio"
          name="gender"
          value="male"
          checked={survey.gender === 'male'}
          onChange={handleChange}
        />
        Male
      </label>
      <label>
        <input
          type="radio"
          name="gender"
          value="female"
          checked={survey.gender === 'female'}
          onChange={handleChange}
        />

```

```

        Female
    </label>
</div>

{/* Multiple Checkboxes */}
<div>
    <label>
        <input
            type="checkbox"
            value="sports"
            checked={survey.interests.includes('sports')}
            onChange={handleCheckboxChange}
        />
        Sports
    </label>
    <label>
        <input
            type="checkbox"
            value="music"
            checked={survey.interests.includes('music')}
            onChange={handleCheckboxChange}
        />
        Music
    </label>
    <label>
        <input
            type="checkbox"
            value="reading"
            checked={survey.interests.includes('reading')}
            onChange={handleCheckboxChange}
        />
        Reading
    </label>
</div>

{/* Textarea */}
<textarea
    name="feedback"
    value={survey.feedback}
    onChange={handleChange}
    placeholder="Your feedback..."
    rows="4"
/>

{/* Range Input */}
<div>
    <label>Rating: {survey.rating}</label>
    <input
        type="range"
        name="rating"
        min="1"
        max="10"
        value={survey.rating}
    />

```

```

        onChange={handleChange}
      />
    </div>

    <button type="submit">Submit Survey</button>
  </form>
);
}

```

Basic Validation

Simple Validation

```

function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});

  function validate() {
    const newErrors = {};

    // Email validation
    if (!email) {
      newErrors.email = 'Email is required';
    } else if (!/\S+@\S+\.\S+/.test(email)) {
      newErrors.email = 'Email is invalid';
    }

    // Password validation
    if (!password) {
      newErrors.password = 'Password is required';
    } else if (password.length < 6) {
      newErrors.password = 'Password must be at least 6 characters';
    }

    return newErrors;
  }

  function handleSubmit(e) {
    e.preventDefault();

    const validationErrors = validate();

    if (Object.keys(validationErrors).length > 0) {
      setErrors(validationErrors);
    } else {
      setErrors({});
      console.log('Form is valid!', { email, password });
    }
  }
}

```

```

    // Submit form
  }
}

return (
  <form onSubmit={handleSubmit}>
    <div>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      {errors.email && <p style={{ color: 'red' }}>{errors.email}</p>}
    </div>

    <div>
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      {errors.password && <p style={{ color: 'red' }}>{errors.password}</p>}
    </div>

    <button type="submit">Login</button>
  </form>
);
}

```

Real-time Validation

```

function SignupForm() {
  const [username, setUsername] = useState('');
  const [usernameError, setUsernameError] = useState('');

  function validateUsername(value) {
    if (value.length < 3) {
      setUsernameError('Username must be at least 3 characters');
    } else if (value.length > 15) {
      setUsernameError('Username must be less than 15 characters');
    } else if (!/^[a-zA-Z0-9_]+$/.test(value)) {
      setUsernameError('Username can only contain letters, numbers, and underscores');
    } else {
      setUsernameError('');
    }
  }

  function handleChange(e) {
    const value = e.target.value;

```

```

    setUsername(value);
    validateUsername(value);
  }

  return (
    <div>
      <input
        value={username}
        onChange={handleChange}
        placeholder="Username"
        className={usernameError ? 'error' : ''}
      />
      {usernameError && <p style={{ color: 'red' }}>{usernameError}</p>}
      {!usernameError && username && <p style={{ color: 'green' }}>✓ Valid username</p>}
    </div>
  );
}

```

Password Strength Indicator

```

function PasswordInput() {
  const [password, setPassword] = useState('');
  const [strength, setStrength] = useState('');

  function checkPasswordStrength(pwd) {
    if (pwd.length < 6) {
      return 'weak';
    } else if (pwd.length < 10) {
      return 'medium';
    } else if (/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])/.test(pwd)) {
      return 'strong';
    } else {
      return 'medium';
    }
  }

  function handleChange(e) {
    const pwd = e.target.value;
    setPassword(pwd);
    setStrength(checkPasswordStrength(pwd));
  }

  function getStrengthColor() {
    switch(strength) {
      case 'weak': return 'red';
      case 'medium': return 'orange';
      case 'strong': return 'green';
      default: return 'gray';
    }
  }
}

```



```

return (
  <div>
    <input
      type="password"
      value={password}
      onChange={handleChange}
      placeholder="Enter password"
    />
    {password && (
      <div>
        <div
          style={{
            height: '5px',
            width: '100%',
            backgroundColor: '#ddd',
            marginTop: '5px'
          }}
        >
          <div
            style={{
              height: '100%',
              width: strength === 'weak' ? '33%' : strength === 'medium' ? '66%' : '100%',
              backgroundColor: getStrengthColor(),
              transition: 'all 0.3s'
            }}
          />
        </div>
        <p style={{ color: getStrengthColor() }}>
          Password Strength: {strength}
        </p>
      </div>
    )}
  </div>
);
}

```



Form Submission

Basic Form Submission

```

function ContactForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: ''
  });
  const [submitted, setSubmitted] = useState(false);

```

```

function handleChange(e) {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
}

function handleSubmit(e) {
  e.preventDefault();

  // Simulate API call
  console.log('Submitting:', formData);

  // Reset form
  setFormData({
    name: '',
    email: '',
    message: ''
  });

  setSubmitted(true);

  // Hide success message after 3 seconds
  setTimeout(() => setSubmitted(false), 3000);
}

return (
  <div>
    {submitted && (
      <div style={{
        padding: '10px',
        backgroundColor: 'lightgreen',
        marginBottom: '10px'
      }}>
        ✓ Form submitted successfully!
      </div>
    )}

    <form onSubmit={handleSubmit}>
      <input
        name="name"
        value={formData.name}
        onChange={handleChange}
        placeholder="Your Name"
        required
      />

      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}

```

```

        placeholder="Your Email"
        required
    />

    <textarea
        name="message"
        value={formData.message}
        onChange={handleChange}
        placeholder="Your Message"
        required
    />

    <button type="submit">Send Message</button>
</form>
</div>
);
}

```

Async Form Submission

```

function AsyncForm() {
  const [formData, setFormData] = useState({ username: '', email: '' });
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(false);

  async function handleSubmit(e) {
    e.preventDefault();
    setIsSubmitting(true);
    setError('');
    setSuccess(false);

    try {
      // Simulate API call
      await new Promise((resolve, reject) => {
        setTimeout(() => {
          // Simulate random success/failure
          if (Math.random() > 0.2) {
            resolve();
          } else {
            reject(new Error('Submission failed'));
          }
        }, 2000);
      });

      setSuccess(true);
      setFormData({ username: '', email: '' });
    } catch (err) {
      setError(err.message || 'Something went wrong');
    } finally {
      setIsSubmitting(false);
    }
  }
}

```

```

    }
  }

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="username"
        value={formData.username}
        onChange={handleChange}
        disabled={isSubmitting}
        required
      />

      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        disabled={isSubmitting}
        required
      />

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Submitting...' : 'Submit'}
      </button>

      {error && <p style={{ color: 'red' }}>✖ {error}</p>}
      {success && <p style={{ color: 'green' }}>✓ Submitted successfully!</p>}
    </form>
  );
}

```

Form UI State

Loading States

```

function FormWithLoading() {
  const [formData, setFormData] = useState({ email: '' });
  const [isLoading, setIsLoading] = useState(false);

```

```

async function handleSubmit(e) {
  e.preventDefault();
  setIsLoading(true);

  // Simulate API call
  await new Promise(resolve => setTimeout(resolve, 2000));

  setIsLoading(false);
  alert('Form submitted!');
}

return (
  <form onSubmit={handleSubmit}>
    <input
      type="email"
      value={formData.email}
      onChange={(e) => setFormData({ email: e.target.value })}
      disabled={isLoading}
      placeholder="Enter email"
    />

    <button type="submit" disabled={isLoading}>
      {isLoading ? (
        <>
          <span className="spinner"> ⌚ </span> Loading...
        </>
      ) : (
        'Submit'
      )}
    </button>
  </form>
);
}

```

Success and Error States

```

function FormWithStates() {
  const [formData, setFormData] = useState({ email: '' });
  const [status, setStatus] = useState('idle'); // idle, loading, success, error
  const [message, setMessage] = useState('');

  async function handleSubmit(e) {
    e.preventDefault();
    setStatus('loading');

    try {
      // Simulate API call
      await new Promise((resolve, reject) => {
        setTimeout(() => {
          if (formData.email.includes('@')) {
            resolve();
          }
        }, 2000);
      });
    }
  }
}

```

```

    } else {
      reject(new Error('Invalid email'));
    }
  }, 1500);
});

setStatus('success');
setMessage('Email subscribed successfully!');
setFormData({ email: '' });
} catch (error) {
  setStatus('error');
  setMessage(error.message);
}
}

return (
  <div>
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={formData.email}
        onChange={(e) => setFormData({ email: e.target.value })}
        disabled={status === 'loading'}
        placeholder="Enter your email"
      />

      <button type="submit" disabled={status === 'loading'}>
        {status === 'loading' ? 'Subscribing...' : 'Subscribe'}
      </button>
    </form>

    {status === 'success' && (
      <div style={{
        padding: '10px',
        backgroundColor: '#d4edda',
        color: '#155724',
        marginTop: '10px'
      }}>
        ✓ {message}
      </div>
    )}

    {status === 'error' && (
      <div style={{
        padding: '10px',
        backgroundColor: '#f8d7da',
        color: '#721c24',
        marginTop: '10px'
      }}>
        ✗ {message}
      </div>
    )}
  </div>

```

```
);  
}
```

Disabled State Management

```
function SmartForm() {  
  const [formData, setFormData] = useState({  
    firstName: '',  
    lastName: '',  
    email: '',  
    agreeToTerms: false  
  });  
  
  function handleChange(e) {  
    const { name, value, type, checked } = e.target;  
    setFormData(prev => ({  
      ...prev,  
      [name]: type === 'checkbox' ? checked : value  
    }));  
  }  
  
  // Check if form is valid  
  const isFormValid =  
    formData.firstName.trim() !== '' &&  
    formData.lastName.trim() !== '' &&  
    formData.email.includes('@') &&  
    formData.agreeToTerms;  
  
  return (  
    <form>  
      <input  
        name="firstName"  
        value={formData.firstName}  
        onChange={handleChange}  
        placeholder="First Name"  
      />  
  
      <input  
        name="lastName"  
        value={formData.lastName}  
        onChange={handleChange}  
        placeholder="Last Name"  
      />  
  
      <input  
        type="email"  
        name="email"  
        value={formData.email}  
        onChange={handleChange}  
        placeholder="Email"  
      />  
    </form>  
  );  
}
```

```

<label>
  <input
    type="checkbox"
    name="agreeToTerms"
    checked={formData.agreeToTerms}
    onChange={handleChange}
  />
  I agree to the terms and conditions
</label>

<button
  type="submit"
  disabled={!isFormValid}
  style={{
    opacity: isFormValid ? 1 : 0.5,
    cursor: isFormValid ? 'pointer' : 'not-allowed'
  }}
>
  Submit
</button>

{!isFormValid && (
  <p style={{ color: 'gray', fontSize: '12px' }}>
    Please fill all fields and accept terms to continue
  </p>
)}
</form>
);
}

```

Prop Drilling & Lifting Up State

What is Prop Drilling?

Prop drilling is passing props through multiple component levels to reach a deeply nested component.

```

// ❌ Prop Drilling Problem
function App() {
  const [user, setUser] = useState({ name: 'Alice', age: 25 });

  return <Parent user={user} setUser={setUser} />;
}

function Parent({ user, setUser }) {
  return <Child user={user} setUser={setUser} />;
}

```



```

function Child({ user, setUser }) {
  return <GrandChild user={user} setUser={setUser} />;
}

function GrandChild({ user, setUser }) {
  return (
    <div>
      <p>{user.name}</p>
      <button onClick={() => setUser({ ...user, age: user.age + 1 })}>
        Increment Age
      </button>
    </div>
  );
}

```

↑ Lifting State Up

When multiple components need to share state, move the state to their closest common parent.

```

function App() {
  const [temperature, setTemperature] = useState(0);

  return (
    <div>
      <TemperatureInput
        value={temperature}
        onChange={setTemperature}
      />
      <TemperatureDisplay
        celsius={temperature}
      />
    </div>
  );
}

function TemperatureInput({ value, onChange }) {
  return (
    <div>
      <label>Temperature (Celsius):</label>
      <input
        type="number"
        value={value}
        onChange={(e) => onChange(Number(e.target.value))}
      />
    </div>
  );
}

function TemperatureDisplay({ celsius }) {
  const fahrenheit = (celsius * 9/5) + 32;

```

```

return (
  <div>
    <p>{celsius}°C = {fahrenheit.toFixed(1)}°F</p>
    <p>{celsius} >= 100 ? '🔥 Boiling!' : celsius <= 0 ? '❄️ Freezing!' : '🌡️ Normal'</p>
  </div>
);
}

```

Sharing State Between Siblings

```

function ShoppingApp() {
  const [cart, setCart] = useState([]);

  function addToCart(item) {
    setCart([...cart, item]);
  }

  function removeFromCart(index) {
    setCart(cart.filter((_, i) => i !== index));
  }

  return (
    <div>
      <ProductList onAddToCart={addToCart} />
      <Cart items={cart} onRemove={removeFromCart} />
    </div>
  );
}

function ProductList({ onAddToCart }) {
  const products = [
    { id: 1, name: 'Laptop', price: 1000 },
    { id: 2, name: 'Phone', price: 500 }
  ];

  return (
    <div>
      <h2>Products</h2>
      {products.map(product => (
        <div key={product.id}>
          <p>{product.name} - ${product.price}</p>
          <button onClick={() => onAddToCart(product)}>
            Add to Cart
          </button>
        </div>
      ))}
    </div>
  );
}

```

```
function Cart({ items, onRemove }) {
  const total = items.reduce((sum, item) => sum + item.price, 0);

  return (
    <div>
      <h2>Cart ({items.length})</h2>
      {items.map((item, index) => (
        <div key={index}>
          <p>{item.name} - ${item.price}</p>
          <button onClick={() => onRemove(index)}>Remove</button>
        </div>
      ))}
      <h3>Total: ${total}</h3>
    </div>
  );
}
```

💡 Solutions to Prop Drilling

1. **Component Composition** - Pass components as children
2. **Context API** - Share state globally (advanced topic)
3. **State Management Libraries** - Redux, Zustand (advanced topics)

```
// ✅ Better: Using Composition
function App() {
  const [user, setUser] = useState({ name: 'Alice' });

  return (
    <Layout>
      <UserProfile user={user} onUpdate={setUser} />
    </Layout>
  );
}

function Layout({ children }) {
  return (
    <div className="layout">
      <Header />
      {children}
      <Footer />
    </div>
  );
}
```

🎓 Best Practices Summary

✓ Do's

- ✓ Keep components small and focused
- ✓ Use meaningful variable and component names
- ✓ Always provide keys for lists
- ✓ Use controlled components for forms
- ✓ Validate user input
- ✓ Handle loading and error states
- ✓ Use functional updates when new state depends on old state
- ✓ Keep state as local as possible
- ✓ Use composition over prop drilling

✗ Don'ts

- ✗ Don't mutate state directly
- ✗ Don't use index as key for dynamic lists
- ✗ Don't forget to prevent default form submission
- ✗ Don't put hooks inside conditions or loops
- ✗ Don't pass too many props (consider composition)
- ✗ Don't forget to handle edge cases and errors

Happy Coding! 🎉